```c
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<iostream>
// #include<io.h>

#define MAXINODE 50

#define READ 1
#define WRITE 2

#define MAXFILESIZE 2048

#define REGULAR 1
#define SPECIAL 2

#define START 0
#define CURRENT 1
#define END 2

typedef struct superblock
{
    int TotalInodes;
    int FreeInode;
}SUPERBLOCK, *PSUPERBLOCK;

typedef struct inode
{
    char FileName[50];
    int InodeNumber;
    int FileSize;
    int FileActualSize;
    int FileType;
    char *Buffer;
    int LinkCount;
    int ReferenceCount;
    int permission; // 1 2 3
    struct inode *next;
}INODE, *PINODE, **PPINODE;

```

```
42  typedef struct filetable
43  {
44      int readoffset;
45      int writeoffset;
46      int count;
47      int mode;   // 1 2 3
48      PINODE ptrinode;
49  }FILETABLE, *PFILETABLE;
50
51  typedef struct ufdt
52  {
53      PFILETABLE ptrfiletable;
54  }UFDT;
55
56  UFDT UFDTArr[50];
57  SUPERBLOCK SUPERBLOCKObj;
58  PINODE head = NULL;
59
60  void man(char *name)
61  {
62      if(name == NULL) return;
63
64      if(strcmp(name,"create") == 0)
65      {
66          printf("Description : Used to create new regular file\n");
67          printf("Usage : create File_name Permission\n");
68      }
69      else if(strcmp(name,"read") == 0)
70      {
71          printf("Description : Used to read data from regular file\n");
72          printf("Usage : read File_name No_Of_Bytes_To_Read\n");
73      }
74      else if(strcmp(name,"write") == 0)
75      {
76          printf("Description : Used to write into regular file\n");
77          printf("Usage : write File_name\n After this enter the data that we want to write\n");
78      }
79      else if(strcmp(name,"ls") == 0)
80      {
81          printf("Description : Used to list all information of files\n");
82          printf("Usage : ls\n");
83      }
84      else if(strcmp(name,"stat") == 0)
```

```
85      {
86          printf("Description : Used to display information of file\n");
87          printf("Usage : stat File_name\n");
88      }
89      else if(strcmp(name,"fstat") == 0)
90      {
91          printf("Description : Used to display information of file\n");
92          printf("Usage : stat File_Descriptor\n");
93      }
94      else if(strcmp(name,"truncate") == 0)
95      {
96          printf("Description : Used to remove data from file\n");
97          printf("Usage : truncate File_name\n");
98      }
99      else if(strcmp(name,"open") == 0)
100     {
101         printf("Description : Used to open existing file\n");
102         printf("Usage : open File_name mode\n");
103     }
104     else if(strcmp(name,"close") == 0)
105     {
106         printf("Description : Used to close opened file\n");
107         printf("Usage : close File_name\n");
108     }
109     else if(strcmp(name,"closeall") == 0)
110     {
111         printf("Description : Used to close all opened file\n");
112         printf("Usage : closeall\n");
113     }
114     else if(strcmp(name,"lseek") == 0)
115     {
116         // continues...
117         printf("Description : Used to change file offset\n");
118         printf("Usage : lseek File_Name ChangeInOffset StartPoint\n");
119     }
120     else if(strcmp(name,"rm") == 0)
121     {
122         printf("Description : Used to delete the file\n");
123         printf("Usage : rm File_Name\n");
124     }
125     else
126     {
127         printf("ERROR : No manual entry available.\n");
128     }
129 }
130
131 void DisplayHelp()
132 {
133     printf("ls : To list out all files\n");
```

```
134         printf("clear : To clear console\n");
135         printf("open : To open the file\n");
136         printf("close : To close the file\n");
137         printf("closeall : To close all opened files\n");
138         printf("read : To read the contents from file\n");
139         printf("write : To write contents into file\n");
140         printf("exit : To terminate file system\n");
141         printf("stat : To display information of file using name\n");
142         printf("fstat : To display information of file using file descriptor\n");
143         printf("truncate : To remove all data from file\n");
144         printf("rm : To delete the file\n");
145     }
146
147     int GetFDFromName(char *name)
148     {
149         int i = 0;
150
151         while(i < 50)
152         {
153             if(UFDTArr[i].ptrfiletable != NULL)
154                 if(strcmp((UFDTArr[i].ptrfiletable->ptrinode->FileName), name) == 0)
155                     break;
156             i++;
157         }
158
159         if(i == 50) return -1;
160         else  return i;
161     }
162
163     PINODE Get_Inode(char *name)
164     {
165         PINODE temp = head;
166         int i = 0;
167
168         if(name == NULL)
169             return NULL;
170
171         while(temp != NULL)
172         {
173             if(strcmp(name, temp->FileName) == 0)
174                 break;
175             temp = temp->next;
176         }
177         return temp;
178     }
179
180     void CreateDILB()
181     {
182         int i = 1;
```

```
183         PINODE newn = NULL;
184         PINODE temp = head;
185
186         while(i <= MAXINODE)
187         {
188             newn = (PINODE)malloc(sizeof(INODE));
189
190             newn->LinkCount = 0;
191             newn->ReferenceCount = 0;
192             newn->FileType = 0;
193             newn->FileSize = 0;
194
195             newn->Buffer = NULL;
196             newn->next = NULL;
197
198             newn->InodeNumber = i;
199
200             if(temp == NULL)
201             {
202                 head = newn;
203                 temp = head;
204             }
205             else
206             {
207                 temp->next = newn;
208                 temp = temp->next;
209             }
210             i++;
211         }
212         printf("DILB created successfully\n");
213     }
214
215     void InitialiseSuperBlock()
216     {
217         int i = 0;
218         while(i < MAXINODE)
219         {
220             UFDTArr[i].ptrfiletable = NULL;
221             i++;
222         }
223
224         SUPERBLOCKObj.TotalInodes = MAXINODE;
225         SUPERBLOCKObj.FreeInode = MAXINODE;
226     }
227
228     int CreateFile(char *name, int permission)
229     {
230         int i = 0;
231         PINODE temp = head;
232
233         if((name == NULL) || (permission == 0) || (permission > 3))
```

```
234         return -1;
235
236     if(SUPERBLOCKObj.FreeInode == 0) return -2;
237
238     (SUPERBLOCKObj.FreeInode)--;
239
240     if(Get_Inode(name) != NULL)
241         return -3;
242
243     while(temp!= NULL)
244     {
245         if(temp->FileType == 0)
246             break;
247         temp=temp->next;
248     }
249
250     while(i<50)
251     {
252         if(UFDTArr[i].ptrfiletable == NULL)
253             break;
254         i++;
255     }
256
257     UFDTArr[i].ptrfiletable = (PFILETABLE)malloc(sizeof(FILETABLE));
258     UFDTArr[i].ptrfiletable->count = 1;
259     UFDTArr[i].ptrfiletable->mode = permission;
260     UFDTArr[i].ptrfiletable->readoffset = 0;
261     UFDTArr[i].ptrfiletable->writeoffset = 0;
262     UFDTArr[i].ptrfiletable->ptrinode = temp;
263
264     strcpy(UFDTArr[i].ptrfiletable->ptrinode->FileName,name);
265     UFDTArr[i].ptrfiletable->ptrinode->FileType = REGULAR;
266     UFDTArr[i].ptrfiletable->ptrinode->ReferenceCount = 1;
267     UFDTArr[i].ptrfiletable->ptrinode->LinkCount = 1;
268     UFDTArr[i].ptrfiletable->ptrinode->FileSize = MAXFILESIZE;
269     UFDTArr[i].ptrfiletable->ptrinode->FileActualSize = 0;
270     UFDTArr[i].ptrfiletable->ptrinode->permission = permission;
271     UFDTArr[i].ptrfiletable->ptrinode->Buffer = (char *)malloc(MAXFILESIZE);
272
273 return i;
274 }
275
276 // rm_File("Demo.txt")
277 int rm_File(char * name)
278 {
279     int fd = 0;
280     fd = GetFDFromName(name);
281     if(fd == -1)
282         return -1;
283
284     (UFDTArr[fd].ptrfiletable->ptrinode->LinkCount)--;
```

```c
    if(UFDTArr[fd].ptrfiletable->ptrinode->LinkCount == 0)
    {
        UFDTArr[fd].ptrfiletable->ptrinode->FileType = 0;
        // free(UFDTArr[fd].ptrfiletable->ptrinode->Buffer);
        free(UFDTArr[fd].ptrfiletable);
    }
    UFDTArr[fd].ptrfiletable = NULL;
    (SUPERBLOCKObj.FreeInode)++;
}

int ReadFile(int fd, char *arr, int isize)
{
    int read_size = 0;
    if(UFDTArr[fd].ptrfiletable == NULL) return -1;
    if(UFDTArr[fd].ptrfiletable->mode != READ && UFDTArr[fd].ptrfiletable->mode != READ+WRITE) return -2;
    if(UFDTArr[fd].ptrfiletable->ptrinode->permission != READ && UFDTArr[fd].ptrfiletable->ptrinode->permission != READ+WRITE) return -2;
    if(UFDTArr[fd].ptrfiletable->readoffset == UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) return -3;
    if(UFDTArr[fd].ptrfiletable->ptrinode->FileType != REGULAR) return -4;

    read_size = (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) - (UFDTArr[fd].ptrfiletable->readoffset);

    if(read_size < isize)
    {
        strncpy(arr, (UFDTArr[fd].ptrfiletable->ptrinode->Buffer) + (UFDTArr[fd].ptrfiletable->readoffset), read_size);
        UFDTArr[fd].ptrfiletable->readoffset = UFDTArr[fd].ptrfiletable->readoffset + read_size;
    }
    else
    {
        strncpy(arr, (UFDTArr[fd].ptrfiletable->ptrinode->Buffer) + (UFDTArr[fd].ptrfiletable->readoffset), isize);
        UFDTArr[fd].ptrfiletable->readoffset = (UFDTArr[fd].ptrfiletable->readoffset) + isize;
    }
    return isize;
}

int WriteFile(int fd, char *arr, int isize)
{
    if(((UFDTArr[fd].ptrfiletable->mode) != WRITE) && ((UFDTArr[fd].ptrfiletable->mode) != READ+WRITE)) return -1;
    if((UFDTArr[fd].ptrfiletable->ptrinode->permission != WRITE) && (UFDTArr[fd].ptrfiletable->ptrinode->permission != READ+WRITE)) return -1;
    if(UFDTArr[fd].ptrfiletable->writeoffset == MAXFILESIZE) return -2;
    if(UFDTArr[fd].ptrfiletable->ptrinode->FileType != REGULAR) return -3;
    strncpy((UFDTArr[fd].ptrfiletable->ptrinode->Buffer) + (UFDTArr[fd].ptrfiletable->writeoffset), arr, isize);
    (UFDTArr[fd].ptrfiletable->writeoffset) = (UFDTArr[fd].ptrfiletable->writeoffset) + isize;
    (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) = UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize + isize;
return isize;
}

int OpenFile(char *name, int mode)
{
    int i = 0;
    PINODE temp = NULL;
    if(name == NULL || mode <= 0)
```

```
336             return -1;
337         temp = Get_Inode(name);
338         if(temp == NULL)
339             return -2;
340         if(temp->permission < mode)
341             return -3;
342         while(i < 50)
343         {
344             if(UFDTArr[i].ptrfiletable == NULL)
345                 break;
346             i++;
347         }
348         UFDTArr[i].ptrfiletable = (PFILETABLE)malloc(sizeof(FILETABLE));
349         if(UFDTArr[i].ptrfiletable == NULL)
350             return -1;
351         UFDTArr[i].ptrfiletable->count = 1;
352         UFDTArr[i].ptrfiletable->mode = mode;
353         if(mode == READ + WRITE)
354         {
355             UFDTArr[i].ptrfiletable->readoffset = 0;
356             UFDTArr[i].ptrfiletable->writeoffset = 0;
357         }
358         else if(mode == READ)
359         {
360             UFDTArr[i].ptrfiletable->readoffset = 0;
361         }
362         else if(mode == WRITE)
363         {
364             UFDTArr[i].ptrfiletable->writeoffset = 0;
365         }
366         UFDTArr[i].ptrfiletable->ptrinode = temp;
367         (UFDTArr[i].ptrfiletable->ptrinode->ReferenceCount)++;
368         return i;
369     }
370
371     // // int CloseFileByName(int fd)
372     // {
373     //     UFDTArr[fd].ptrfiletable->readoffset = 0;
374     //     UFDTArr[fd].ptrfiletable->writeoffset = 0;
375     //     UFDTArr[fd].ptrfiletable->ptrinode->ReferenceCount--;
376     // }
377
378     int CloseFileByName(char *name)
379     {
380         int i = 0;
381         i = GetFDFromName(name);
382         if(i== -1)
383         return -1;
384
385         UFDTArr[i].ptrfiletable->readoffset = 0;
386         UFDTArr[i].ptrfiletable->writeoffset = 0;
```

```
387         (UFDTArr[i].ptrfiletable->ptrinode->ReferenceCount)--;
388
389     return 0;
390     }
391
392     void CloseAllFile()
393     {
394         int i=0;
395         while(i<50)
396         {
397             if(UFDTArr[i].ptrfiletable != NULL)
398             {
399                 UFDTArr[i].ptrfiletable->readoffset = 0;
400                 UFDTArr[i].ptrfiletable->writeoffset = 0;
401                 (UFDTArr[i].ptrfiletable->ptrinode->ReferenceCount)--;
402                 break;
403             }
404         i++;
405         }
406     }
407
408     int LseekFile(int fd, int size, int from)
409     {
410         if((fd<0) || (from > 2)) return -1;
411         if(UFDTArr[fd].ptrfiletable == NULL) return -1;
412
413         if((UFDTArr[fd].ptrfiletable->mode == READ) || (UFDTArr[fd].ptrfiletable->mode == READ+WRITE))
414         {
415             if(from == CURRENT)
416             {
417                 if(((UFDTArr[fd].ptrfiletable->readoffset) + size) > UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) return -1;
418                 if(((UFDTArr[fd].ptrfiletable->readoffset) + size) < 0) return -1;
419                 (UFDTArr[fd].ptrfiletable->readoffset) = (UFDTArr[fd].ptrfiletable->readoffset) + size;
420             }
421             else if(from == START)
422             {
423                 if(size > (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize)) return -1;
424                 if(size < 0) return -1;
425                 (UFDTArr[fd].ptrfiletable->readoffset) = size;
426             }
427             else if(from == END)
428             {
429                 if((UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) + size > MAXFILESIZE) return -1;
430                 if(((UFDTArr[fd].ptrfiletable->readoffset) + size) < 0) return -1;
431                 (UFDTArr[fd].ptrfiletable->readoffset) = (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) + size;
432             }
433         }
434         else if(UFDTArr[fd].ptrfiletable->mode == WRITE)
435         {
436             if(from == CURRENT)
437             {
```

```
438             if(((UFDTArr[fd].ptrfiletable->writeoffset) + size) > MAXFILESIZE) return -1;
439             if(((UFDTArr[fd].ptrfiletable->writeoffset) + size) < 0) return -1;
440             if(((UFDTArr[fd].ptrfiletable->writeoffset) + size) > (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize))
441                 (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) =
442             (UFDTArr[fd].ptrfiletable->writeoffset) + size;
443                 (UFDTArr[fd].ptrfiletable->writeoffset) = (UFDTArr[fd].ptrfiletable->writeoffset) +
444             size;
445         }
446         else if(from == START)
447         {
448             if(size > MAXFILESIZE) return -1;
449             if(size < 0) return -1;
450             if(size > (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize))
451                 (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) = size;
452                 (UFDTArr[fd].ptrfiletable->writeoffset) = size;
453         }
454         else if(from == END)
455         {
456             if((UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) + size > MAXFILESIZE)
457             return -1;
458             if(((UFDTArr[fd].ptrfiletable->writeoffset) + size) < 0) return -1;
459             (UFDTArr[fd].ptrfiletable->writeoffset) = (UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize) + size;
460         }
461     }
462 }
463
464 void Ls_file()
465 {
466     int i= 0;
467     PINODE temp = head;
468     if (SUPERBLOCKObj.FreeInode == MAXINODE)
469     {
470         printf("Error: There are no files\n");
471         return;
472     }
473     printf("\nFile Name\tInode number\tFile size\tLink count\n");
474     printf("------------------------------------------------------------------\n");
475
476     while(temp != NULL)
477     {
478         if(temp->FileType != 0)
479         {
480             printf("%s\t\t%d\t\t%d\t\t%d\n", temp->FileName, temp->InodeNumber, temp->FileActualSize, temp->LinkCount);
481         }
482     temp = temp->next;
483     }
484     printf("------------------------------------------------------------------\n");
485 }
486
487 int fstat_file(int fd)
488 {
```

```
489         PINODE temp = head;
490         int i = 0;
491         if(fd < 0) return -1;
492         if(UFDTArr[fd].ptrfiletable == NULL) return -2;
493
494         temp = UFDTArr[fd].ptrfiletable->ptrinode;
495
496         printf("\n-----Statistical Information about file-----\n");
497         printf("File name: %s\n", temp->FileName);
498         printf("Inode Number %d\n",temp->InodeNumber);
499         printf("File size: %d\n", temp->FileSize);
500         printf("Actual File size: %d\n", temp->FileActualSize);
501         printf("Link count: %d\n", temp->LinkCount);
502         printf("Reference count: %d\n", temp->ReferenceCount);
503
504         if(temp->permission == 1)
505         printf("File Permission: Read only\n");
506         else if(temp->permission == 2)
507         printf("File Permission: Write\n");
508         else if(temp->permission == 3)
509         printf("File Permission: Read & Write\n");
510         printf("---------------------------------------------------------\n\n");
511
512     return 0;
513     }
514
515     int stat_file(char *name)
516     {
517         PINODE temp = head;
518         int i = 0;
519
520         if(name == NULL) return -1;
521
522         while(temp!= NULL)
523         {
524         if(strcmp(name, temp->FileName) == 0)
525         break;
526         temp = temp->next;
527         }
528
529         if(temp == NULL) return -2;
530
531         printf("\n-----Statistical Information about file-----\n");
532         printf("File name: %s\n", temp->FileName);
533         printf("Inode Number %d\n", temp->InodeNumber);
534         printf("File size: %d\n", temp->FileSize);
535         printf("Actual File size: %d\n", temp->FileActualSize);
536         printf("Link count: %d\n", temp->LinkCount);
537         printf("Reference count: %d\n",temp->ReferenceCount);
538
539         if(temp->permission == 1)
```

```
540         printf("File Permission: Read only\n");
541         else if(temp->permission == 2)
542         printf("File Permission: Write\n");
543         else if(temp->permission == 3)
544         printf("File Permission: Read & Write\n");
545         printf("-----------------------------------------------------------\n\n");
546
547     return 0;
548     }
549
550     int truncate_File(char *name)
551     {
552         int fd = GetFDFromName(name);
553         if(fd == -1)
554         return -1;
555
556         memset(UFDTArr[fd].ptrfiletable->ptrinode->Buffer, 0, 1024);
557         UFDTArr[fd].ptrfiletable->readoffset = 0;
558         UFDTArr[fd].ptrfiletable->writeoffset = 0;
559         UFDTArr[fd].ptrfiletable->ptrinode->FileActualSize = 0;
560     }
561
562     int main()
563     {
564         char *ptr = NULL;
565         int ret = 0, fd = 0, count = 0;
566         char command[4][80], str[80], arr[1024];
567
568         InitialiseSuperBlock();
569         CreateDILB();
570
571         while(1)
572             {
573                 fflush(stdin);
574                 strcpy(str,"");
575
576                 printf("\n Fabulous NVFS : > ");
577
578                 fgets(str,80,stdin);// scanf("%[^'\n']s",str);
579
580                 count = sscanf(str,"%s %s %s %s", command[0],command[1], command[2], command[3]);
581                     if(count == 1)
582                     {
583                         if(strcmp(command[0],"ls") == 0)
584                         {
585                         Ls_file();
586                         }
587                         else if(strcmp(command[0], "closeall") == 0)
588                         {
589                         CloseAllFile();
590                         printf("All files closed successfully\n");
```

```
591                     continue;
592                     }
593                     else if(strcmp(command[0], "clear") == 0)
594                     {
595                     system("cls");
596                     continue;
597                     }
598                     else if(strcmp(command[0],"help") == 0)
599                     {
600                     DisplayHelp();
601                     continue;
602                     }
603                     else if(strcmp(command[0],"exit") == 0)
604                     {
605                     printf("Closing the Fabulous Nextgen Virtual File System\n");
606                     break;
607                     }
608                     else
609                     {
610                         printf("\nERROR: Command Not Available!!!\n");
611                     continue;
612                     }
613                 }
614                 else if(count == 2)
615                 {
616                     if(strcmp(command[0],"stat") == 0)
617                     {
618                         ret = stat_file(command[1]);
619                         if(ret == -1)
620                         printf("ERROR: Incorrect parameters\n");
621                         if(ret == -2)
622                         printf("ERROR: There is no such file\n");
623                         continue;
624                     }
625                     else if(strcmp(command[0], "fstat") == 0)
626                     {
627                         ret = fstat_file(atoi(command[1]));
628                         if(ret == -1)
629                         printf("ERROR: Incorrect parameters\n");
630                         if(ret == -2)
631                         printf("ERROR: There is no such file\n");
632                         continue;
633                     }
634                     else if(strcmp(command[0],"close") == 0)
635                     {
636                         ret = CloseFileByName(command[1]);
637                         if(ret == -1)
638                         printf("ERROR: There is no such file\n");
639                         continue;
640                     }
641                     else if(strcmp(command[0],"rm") == 0)
```

```
642                     {
643                         ret = rm_File(command[1]);
644                         if(ret == -1)
645                         printf("ERROR: There is no such file\n");
646                         continue;
647                     }
648                     else if(strcmp(command[0],"man") == 0)
649                     {
650                         man(command[1]);
651                     }
652                     else if(strcmp(command[0],"write") == 0)
653                     {
654                         fd = GetFDFromName(command[1]);
655                         if(fd == -1)
656                     {
657                         printf("Error: Incorrect parameter\n");
658                         continue;
659                     }
660                     printf("Enter the data : \n");
661                     scanf("%[^\n]",arr);
662                     ret = strlen(arr);
663                     if(ret == 0)
664                     {
665                         printf("Error: Incorrect parameter\n");
666                         continue;
667                     }
668                     ret = WriteFile(fd,arr,ret);
669                     if(ret == -1)
670                     printf("ERROR: Permission denied\n");
671                     if(ret == -2)
672                     printf("ERROR: There is no sufficient memory to write\n");
673                     if(ret == -3)
674                     printf("ERROR: It is not regular file\n");
675                 }
676                     else if(strcmp(command[0], "truncate") == 0)
677                 {
678                     ret = truncate_File(command[1]);
679                     if(ret == -1)
680                     printf("Error: Incorrect parameter\n");
681                 }
682                     else
683                 {
684                     printf("\nERROR: Command not found !!!\n");
685                     continue;
686                 }
687                 }
688                 else if(count == 3)
689                 {
690                     if(strcmp(command[0], "create") == 0)
691                     {
692                         ret = CreateFile(command[1], atoi(command[2]));
```

```
693                     if(ret >= 0)
694                     printf("File is successfully created with file descriptor: %d\n",ret);
695                     if(ret == -1)
696                     printf("ERROR: Incorrect parameters\n");
697                     if(ret == -2)
698                     printf("ERROR: There is no inodes\n");
699                     if(ret == -3)
700                     printf("ERROR: File already exists\n");
701                     if(ret == -4)
702                     printf("ERROR: Memory allocation failure\n");
703                     continue;
704                 }
705                 else if(strcmp(command[0],"open") == 0)
706                 {
707                     ret = OpenFile(command[1], atoi(command[2]));
708                     if(ret >= 0)
709                     printf("File is successfully opened with file descriptor: %d\n",ret);
710                     if(ret == -1)
711                     printf("ERROR: Incorrect parameters\n");
712                     if(ret == -2)
713                     printf("ERROR: File not present\n");
714                     if(ret == -3)
715                     printf("ERROR: Permission denied\n");
716                     continue;
717                 }
718                 else if(strcmp(command[0],"read") == 0)
719                 {
720                     fd = GetFDFromName(command[1]);
721                     if(fd == -1)
722                 {
723                     printf("Error: Incorrect parameter\n");
724                     continue;
725                 }
726                     ptr = (char *)malloc(sizeof(atoi(command[2]))+1);
727                 if(ptr == NULL)
728                 {
729                     printf("Error: Memory allocation failure\n");
730                     continue;
731                 }
732                 ret = ReadFile(fd, ptr, atoi(command[2]));
733                 if(ret == -1)
734                     printf("ERROR : File not existing\n");
735                 if(ret == -2)
736                     printf("ERROR : Permission denied\n");
737                 if(ret == -3)
738                     printf("ERROR : Reached at end of file\n");
739                 if(ret == -4)
740                     printf("ERROR : It is not regular file\n");
741                 if(ret == 0)
742                     printf("ERROR : File empty\n");
743                 if(ret > 0)
```

```
744                     {
745                         write(2,ptr,ret);
746                     }
747                     continue;
748                     }
749                     else
750                     {
751                         printf("\nERROR : Command not found !!!\n");
752                         continue;
753                     }
754                 }
755                 else if(count == 4)
756                 {
757                     if(strcmp(command[0], "Iseek") == 0)
758                     {
759                         fd = GetFDFromName(command[1]);
760                         if(fd == -1)
761                         {
762                             printf("Error: Incorrect parameter\n");
763                             continue;
764                         }
765                         ret = LseekFile(fd,atoi(command[2]),atoi(command[3]));
766                         if(ret == -1)
767                         {
768                             printf("ERROR: Unable to perform Iseek\n");
769                         }
770                     }
771                     else
772                     {
773                         printf("\nERROR: Command not found !!!\n");
774                         continue;
775                     }
776                 }
777                 else
778     {
779         printf("\nERROR: Command not found !!!\n");
780         continue;
781     }
782         }
783 return 0;
784 }
```